

2025/2026	Projet – Étape 4
BTS SIO	Auteur : Elven LE CORRE-PENEAU, Lucas LE CHAMPION (Groupe 7)
1SIOA	Date de rédaction : 07/04/2026

Compte-rendu d'étape

Etape 4 : Classes d'accès aux données (DAO)

Le code source est structuré selon ce qu'on nomme un design pattern. Il est architecturé en couches, chaque couche ayant son propre rôle :

- Interfaces graphiques : interfaces utilisateurs avec contrôles.
- Classes métiers : structurent les données traitées par le programme (Cf. Diagramme UML des classes métier ci-dessous).
- Classes Dao : c'est une couche intermédiaire entre les classes métier et la base de données ; c'est là que sont exécutées les requêtes SQL
- Pour chaque classe métier, il existe une classe DAO
- Les méthodes qui exécutent un SELECT transforment les données récupérées depuis la base de données en objets métier.
- Les méthodes qui exécutent un INSERT ou un UPDATE utilisent les objets métier pour créer ou modifier les enregistrements de la base de données.

Cette étape consiste à de tester les classes d'accès à la base de données (couche DAO).

Travail à faire

- Testez la classe DaoService.
- Codez et mettez au point les classes DaoCategorie et TestDaoCategorie.
- Si nécessaire, adaptez la classe DaoSalarie pour tenir compte de la catégorie du salarié.
- Codez et mettez au point la classe TestDaoSalarie.

Remarques

- Chaque classe DAO fournit (a minima) les méthodes suivantes :
 - o une méthode `getOneById` qui retourne un objet métier d'après son identifiant ;
 - o une méthode `getAll` qui retourne une collection d'objets métier tirés de l'ensemble des enregistrements de la table correspondante.
- Ces classes peuvent être complétées en fonction des besoins : vous pourrez ajouter les méthodes qui vous semblent nécessaires à l'occasion des étapes suivantes.
- Les classes DAO seront regroupées dans un paquetage `modele.dao` ; leurs noms portent le préfixe « Dao ».
- Les classes de test unitaire seront situées dans le paquetage `test.dao` ; leurs noms portent le préfixe « Test ».
- Le code permettant la connexion à la base de données se trouve dans une classe commune à toutes les classes DAO, afin d'éviter de le dupliquer : `ConnexionBDD` (classe fournie).
- Les paramètres de connexion sont fixés dans le fichier `"sirhJdbc.properties"` (fourni). L'application se connecte avec l'utilisateur `"sirh_util"` qui ne possède de droits que sur ses propres tables.

Dans un premier temps nous allons tester la classe `DaoService` avec `TestDaoService` pour savoir ce qu'elle fait et ce qu'elle retourne quand on l'exécute.

2025/2026	Projet – Étape 4
BTS SIO	Auteur : Elven LE CORRE-PENEAU, Lucas LE CHAMPION (Groupe 7)
1SIOA	Date de rédaction : 07/04/2026

```
Test 1 : DaoService.getOneById
getConnexion : jdbc:mysql://localhost/SIRH
Service d'id 2 trouvé :
Service{code=2, designation=Administration, email=administration@infoware.com, telephone=0169983210}
```

```
Test 2 : DaoService.getAll
Service{code=1, designation=Informatique, email=informatique@infoware.com, telephone=0169983212}
Service{code=2, designation=Administration, email=administration@infoware.com, telephone=0169983210}
Service{code=3, designation=Commercial, email=commercial@infoware.com, telephone=0169983215}
Service{code=4, designation=Comptable, email=comptable@infoware.com, telephone=0169983218}
```

Lorsque on execute on obtient ça, le premier test consiste à se connecter à la base de données puis de rechercher le service d'un id ici le 2.

Pour le deuxième test on liste tout les services présent dans la base de données.

Maintenant nous allons coder les classes DaoCategorie et testDaoCategorie sur les modèles précédents de la classe Service. Il suffit de prendre le code et de changer les variables services par les variables catégories. Dans un premier temps nous faisons la méthode getOneById qui permet de récupérer les informations d'une catégorie qui sont présent dans la base de données et qui sont liées avec l'id choisi ensuite ça renvoi l'intégralité des informations dans la console.

```
public class DaoCategorie {
    public static Categorie getOneById(String id) throws SQLException, IOException, FileNotFoundException, ConnexionBDDException {
        Categorie categorieTrouve = null;
        Connection cnx = ConnexionBDD.getConnexion();
        PreparedStatement pstmt = cnx.prepareStatement("SELECT * FROM Categorie WHERE Code = ?");
        pstmt.setString(1, id);
        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) {
            categorieTrouve = new Categorie(
                id,
                rs.getString("Libelle"),
                rs.getDouble("salaireMini"),
                rs.getString("CaisseDeRetraite"),
                rs.getInt("PrimeResultat")
            );
        }
        return categorieTrouve;
    }
}
```

```
Test 1 : DaoCategorie.getOneById
getConnexion : jdbc:mysql://localhost/SIRH
Categorie d'id C1 trouvé :
Categorie{code=C1, libelle=Cadre moyen, salaireMini=2500.0, caisseRetraite=AGIRC, prime=1}
```

Dans un second temps il y a la méthode getAll qui permet de lister toutes les catégories et leurs caractéristiques respectives. On effectue les test de la classe à l'aide de TestDaoCategorie.

```
/**
 * Extraire l'ensemble des enregistrements de la table CATEGORIE
 * @return liste d'objets de type Categorie
 * @throws SQLException
 * @throws java.io.IOException
 * @throws java.io.FileNotFoundException
 * @throws appli.exceptions.ConnexionBDDException
 */
public static ArrayList<Categorie> getAll() throws SQLException, IOException, FileNotFoundException, ConnexionBDDException {
    ArrayList<Categorie> lesCategoriesTrouves = new ArrayList<>();
    Connection cnx = ConnexionBDD.getConnexion();
    PreparedStatement pstmt = cnx.prepareStatement("SELECT * FROM Categorie");
    ResultSet rs = pstmt.executeQuery();
    while (rs.next()) {
        Categorie uneCategorie = new Categorie(
            rs.getString("Code"),
            rs.getString("Libelle"),
            rs.getDouble("salaireMini"),
            rs.getString("CaisseDeRetraite"),
            rs.getInt("PrimeResultat")
        );
        lesCategoriesTrouves.add(uneCategorie);
    }
    return lesCategoriesTrouves;
}
```

2025/2026	Projet – Étape 4
BTS SIO	Auteur : Elven LE CORRE-PENEAU, Lucas LE CHAMPION (Groupe 7)
1SIOA	Date de rédaction : 07/04/2026

```
Test 2 : DaoCategorie.getAll
Categorie{code=C1, libelle=Cadre moyen, salaireMini=2500.0, caisseRetraite=AGIRC, prime=1}
Categorie{code=C2, libelle=Cadre supérieur, salaireMini=2500.0, caisseRetraite=AGIRC, prime=1}
Categorie{code=E1, libelle=Employé niveau 1, salaireMini=1600.0, caisseRetraite=ARRCO, prime=0}
Categorie{code=E2, libelle=Employé niveau 2, salaireMini=2100.0, caisseRetraite=ARRCO, prime=0}
4 categories trouvées
```

Pour le code de TestDaoCategorie c'est la même chose que TestDaoService sauf que l'on adapte pour la catégorie notamment avec des objets catégories à la place des objets services, il faut aussi faire attention au fait que l'id de catégorie est un String et non pas un entier.

```
// Test 1 getById
System.out.println("\n Test 1 : DaoCategorie.getById");
try {
    String idRecherche = "C1";
    Categorie cat = DaoCategorie.getById(idRecherche);
    if(cat != null){
        System.out.println("Categorie d'id "+idRecherche+" trouvé : \n"+cat.toStringEtat());
    }else{
        System.out.println("Categorie d'id "+idRecherche+" non trouvé : \n");
    }
}
} catch (SQLException ex) {
    System.out.println("TestDaoCategorie - échec getById : " + ex.getMessage());
} catch (IOException ex) {
    System.out.println("TestDaoCategorie - échec getById : " + ex.getMessage());
} catch (ConnexionBDDException ex) {
    System.out.println("TestDaoCategorie - échec de connexion à la BDD : " + ex.getMessage());
}
}
```

On voit ici que l'id recherche est un string.

Nous allons maintenant faire comme avant avec les classe Dao précédentes mais cette fois avec des objets Salaries. Pour cela il suffit de modifier la méthode creerSalarie en ajoutant unService et uneCategorie avec les méthodes getById.

```
private static Salarie creerSalarie(ResultSet rs) throws SQLException, IOException, FileNotFoundException, ConnexionBDDException {
    Salarie unSalarie = null;
    // Récupération du service du salarié
    Service unService = DaoService.getById(rs.getInt("CodeServ"));
    // Récupération de la categorie du salarié
    Categorie uneCategorie = DaoCategorie.getById(rs.getString("NumCat"));
    unSalarie = new Salarie(
        rs.getString("Code"),
        rs.getString("Nom"),
        rs.getString("Prenom"),
        (java.util.Date) rs.getDate("DateNaiss"),
        (java.util.Date) rs.getDate("DateEmbauche"),
        rs.getString("Fonction"),
        rs.getDouble("TauxHoraire"),
        rs.getString("situationFamiliale"),
        rs.getInt("NbrEnfants"),
        unService,
        uneCategorie
    );
    return unSalarie;
}
```

A présent nous allons mettre en place la classe TestDaoSalarie qui comme les précédentes est quasiment identique mais il faut changer les éléments avec les classes, méthodes et attributs du groupe Salarie. Lorsque on exécute cela nous donne un salarié et toutes ses caractéristiques puis une qui donne tous les salarié présent dans la base de données.

2025/2026	Projet – Étape 4
BTS SIO 1SIOA	Auteur : Elven LE CORRE-PENEAU, Lucas LE CHAMPION (Groupe 7)
	Date de rédaction : 07/04/2026

```

Test 1 : DaoSalarie.getOneById
getConnexion : jdbc:mysql://localhost/SIRH
Salarie d'id 801 trouv   :
Salarie{code=801, nom=RETAIS, prenom=Claude, dateNaiss=31/03/1967 , dateEmbauche=01/09/2020 , fonction=Chef de projet, tauxHoraire=32,50, situationFamiliale=Mari  .e, nbrEnfants=3,
service=Service{code=1, designation=Informatique, email=informatique@infoware.com, telephone=0169983212},
categorie=Categorie{code=C2, libelle=Cadre sup  rieur, salaireMini=3500.0, caisseRetraite=AGIRC, prime=1}}

Test 2 : DaoSalarie.getAll
Salarie{code=801, nom=RETAIS, prenom=Claude, dateNaiss=31/03/1967 , dateEmbauche=01/09/2020 , fonction=Chef de projet, tauxHoraire=32,50, situationFamiliale=Mari  .e, nbrEnfants=3,
service=Service{code=1, designation=Informatique, email=informatique@infoware.com, telephone=0169983212},
categorie=Categorie{code=C2, libelle=Cadre sup  rieur, salaireMini=3500.0, caisseRetraite=AGIRC, prime=1}}
Salarie{code=802, nom=BERNARD, prenom=C  line, dateNaiss=14/08/1977 , dateEmbauche=01/09/2020 , fonction=Directrice commerciale, tauxHoraire=32,50, situationFamiliale=Mari  .e, nbrEnfants=2,
service=Service{code=3, designation=Commercial, email=commercial@infoware.com, telephone=0169983215},
categorie=Categorie{code=C2, libelle=Cadre sup  rieur, salaireMini=3500.0, caisseRetraite=AGIRC, prime=1}}
Salarie{code=803, nom=RETAIS, prenom=J  r  me, dateNaiss=14/09/1983 , dateEmbauche=01/09/2020 , fonction=Ing  nieur informatique, tauxHoraire=28,00, situationFamiliale=Divorc  .e, nbrEnfants=4,
service=Service{code=3, designation=Commercial, email=commercial@infoware.com, telephone=0169983215}}

```